Lab 3: Architecture Justification - Self-Service Coffee Kiosk

Aryan Upadhyay | SRN: PES1UG25CS806 | Section C | Software Engineering (UE24CS341A)

Architecture Selection: "We chose a Layered Architecture for the Self-Service Coffee Kiosk System."

Architectural style analysis

Style	Fit for one cafe kiosk	Main drawback here
Layered (chosen)	One app on one device; clear UI / business / data split; easy to test and maintain.	Small call overhead between layers (negligible in-process).
Microservices	Independent scaling and deployment are not needed for 3 drinks and 1 kiosk.	Network hops, deployment and data-consistency complexity with no benefit.
Client-Server	Useful for a chain with a central menu server.	Central server becomes a single point of failure; kiosk stops if the network or server is down.

Components

- **Touch Screen UI** (Presentation): handles touch input and screens; uses IOrder only.
- **Order Manager** (Business, given): builds the order, applies prices, coordinates payment and receipt.
- **Payment Service** (Business, given): talks to the external Payment Gateway; returns only approval + last 4 digits.
- **Receipt Service** (Business): formats the receipt and drives the printer hardware.
- **Menu & Order Repository** (Data): stores menu, prices and paid orders in the kiosk database.

Two reasons for this choice

1. **The kiosk is a single, self-contained application with a small fixed scope** (3 coffee types, 2 sizes, card-only payment, one printer). A layered design gives clean separation between the touch screen, the ordering rules and the stored menu without the operational cost of microservices (network calls, separate deployments, distributed data) that brings no benefit on one device.
2. **Each hardware or external dependency is isolated in one layer.** Touch handling lives only in the Presentation layer, the printer driver only in Receipt Service, the card gateway only in Payment Service, and menu/pricing data only in the Data layer. Replacing the printer model, switching payment provider or changing prices touches one component, and each layer can be tested with the layer below mocked.

Security advantage

Strict layering means the UI can never call the Payment Gateway or the database directly. Card data flows only through Payment Service, which sends it to the gateway over TLS 1.2+ and passes back only an approval code and the last 4 digits. Full card numbers therefore never reach the Order Manager or the Data layer, which satisfies NFR-002 (no stored card data) and keeps the PCI-DSS scope to a single component.

Performance benefit

All layers run in one process on the kiosk, so UI -> Order Manager -> Repository calls are local function calls (microseconds) rather than network requests, and the menu and prices are loaded into memory at start-up. The only network call in an order is the single payment authorization, which keeps the full order comfortably within the 60-second target (NFR-001) even at peak hours.